

PROJECT REPORT

Weather Prediction app (MLOPS)

Software Production Engineering

MASTER OF TECHNOLOGY
in
Computer Science & Engineering

Department of Computer Science and Engineering International Institute
of Information Technology Bangalore - 560100, India April 2025

GitHub Link: https://github.com/girishofficial/Weather_ops

Submitted by
Girish Kumar Sahu (MT2024051)
Tathagata Das (MT2024161)

Table of Contents

Weather_ops: Weather Forecasting System Technical Report:	5
Executive Summary:	5
1. System Architecture	5
1.1 Microservices Architecture	6
1.2 Containerization	6
1.3 Kubernetes Orchestration	6
1.4 Data Flow Architecture	7
1.5 Observability Stack	7
1.6 CI/CD Pipeline	7
2. Technology Stack	7
2.2 Backend Technologies	8
2.3 DevOps & Infrastructure	8
2.4 Data Pipeline	8
2.5 Monitoring & Logging Infrastructure	8
2.6 Testing & Quality Assurance	8
1. Frontend Component	9
1.1 Implementation Details	9
1.2 Key Features and Capabilities	9
1.3 Integration Points	9
2. Backend Component	10
2.1 Implementation Details	10
2.2 Key Features and Capabilities	10
2.3 Integration Points	10
2.4 API Design & Endpoints	10
2.5 Data Processing Pipeline	11
2.6 Machine Learning Implementation	11
2.7 Performance & Scalability	11
2.8 Security Measures	11
2.9 Testing Strategy	12
2.10 Monitoring & Observability	12
Key Components	12
Running the Backend	13
3. Data Processing Pipeline	14
3.1 Implementation Details	14
3.2 Key Features and Capabilities	14
3.3 Integration Points	14
3.4 Data Flow Structure	14

3.5 Data Cleaning Operations	14
4. Machine Learning Implementation.....	16
4.1 Implementation Details	16
4.2 Key Features and Capabilities	16
4.3 Integration Points.....	16
4.4 Model Architecture.....	16
4.5 Training Process	16
4.6 Prediction Pipeline	16
4.7 Performance Monitoring.....	17
5. CI/CD Implementation	18
5.1 Implementation Details	18
5.2 Key Features and Capabilities	18
5.3 Integration Points.....	18
5.4 Pipeline Stages.....	19
5.5 Deployment Strategy.....	19
5.6 Infrastructure Automation	19
5.7 Safe ty Measures	19
6. Security Measures	20
6.1 Container Security.....	20
6.2 Network Security.....	20
6.3 API Security.....	21
6.4 Data Protection.....	21
6.5 Authentication & Authorization	21
6.6 CI/CD Security	21
6.7 Monitoring & Observability	21
7. Kubernetes Deployment and Orchestration.....	21
7.1 Implementation Details	22
7.2 Key Features and Capabilities	22
7.3 Core Application Components.....	22
7.4 Scaling and Resilience	23
7.5 Monitoring Infrastructure	23
7.6 Logging Stack.....	23
8. Monitoring and Logging Infrastructure	25
8.1 Implementation Details	25
8.2 Key Features and Capabilities	26
8.3 Prometheus Setup	26
8.4 Grafana Configuration	26
8.5 Loki Log Aggregation	26

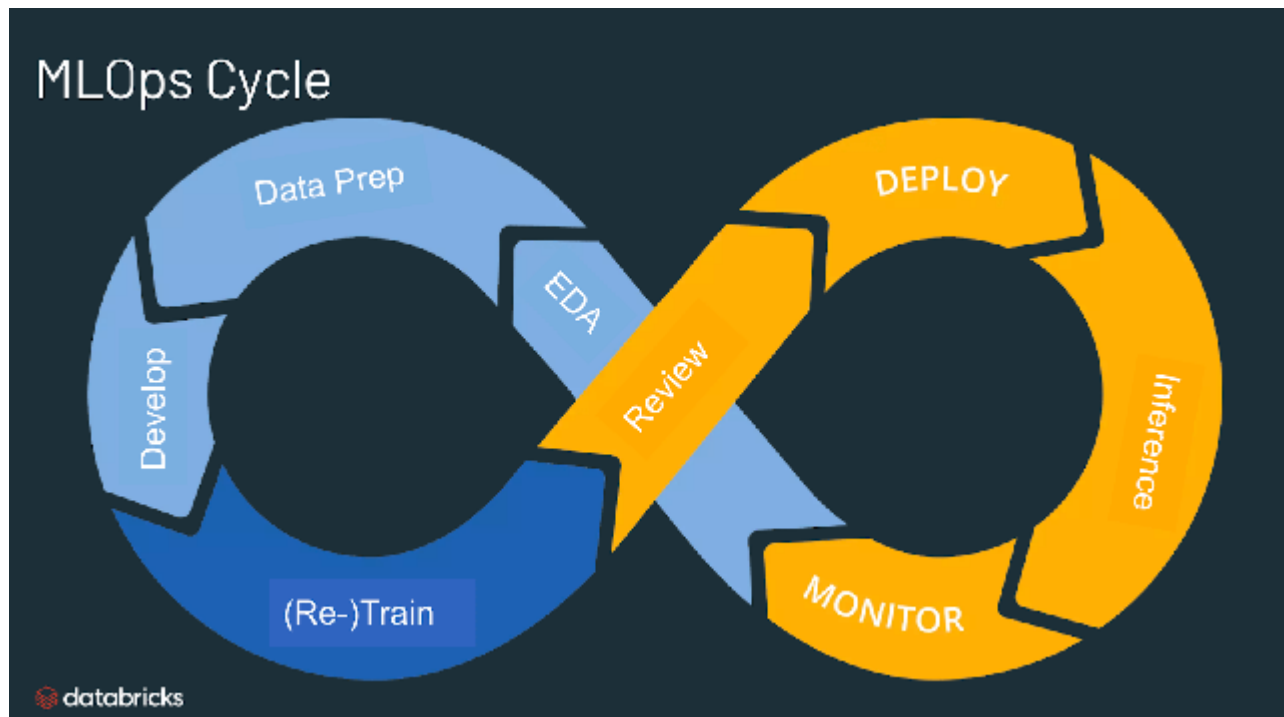
8.6 Promtail Log Collection.....	26
8.7 ELK Stack Alternative.....	26
9. Testing Strategy	28
9.1 Implementation Details	28
9.2 Key Features and Capabilities	28
9.3 Test Categories.....	29
9.4 Testing Approach.....	29
9.5 Test Data Management.....	29
10. Future Enhancements	30
10.1 Machine Learning Improvements	30
10.2 Data Processing Enhancements	30
10.3 Architecture Improvements	30
10.4 Monitoring and Observability	30
10.5 Frontend Experience.....	31
10.6 Operational Improvements	31
10.7 Security Enhancements	31
10.8 Testing Expansion	31
11. Conclusion.....	31
11.1 Project Achievements	31
11.2 Technical Highlights.....	32
11.3 Challenges Overcome.....	32
11.4 Lessons Learned.....	32
11.5 Core Principles Applied	32

Weather_ops: Weather Forecasting System Technical Report:

GitHub Link: https://github.com/girishofficial/Weather_ops

What is MLOps?

MLOps stands for Machine Learning Operations. MLOps is a core function of Machine Learning engineering, focused on streamlining the process of taking machine learning models to production, and then maintaining and monitoring them. MLOps is a collaborative function, often comprising data scientists, devops engineers, and IT.

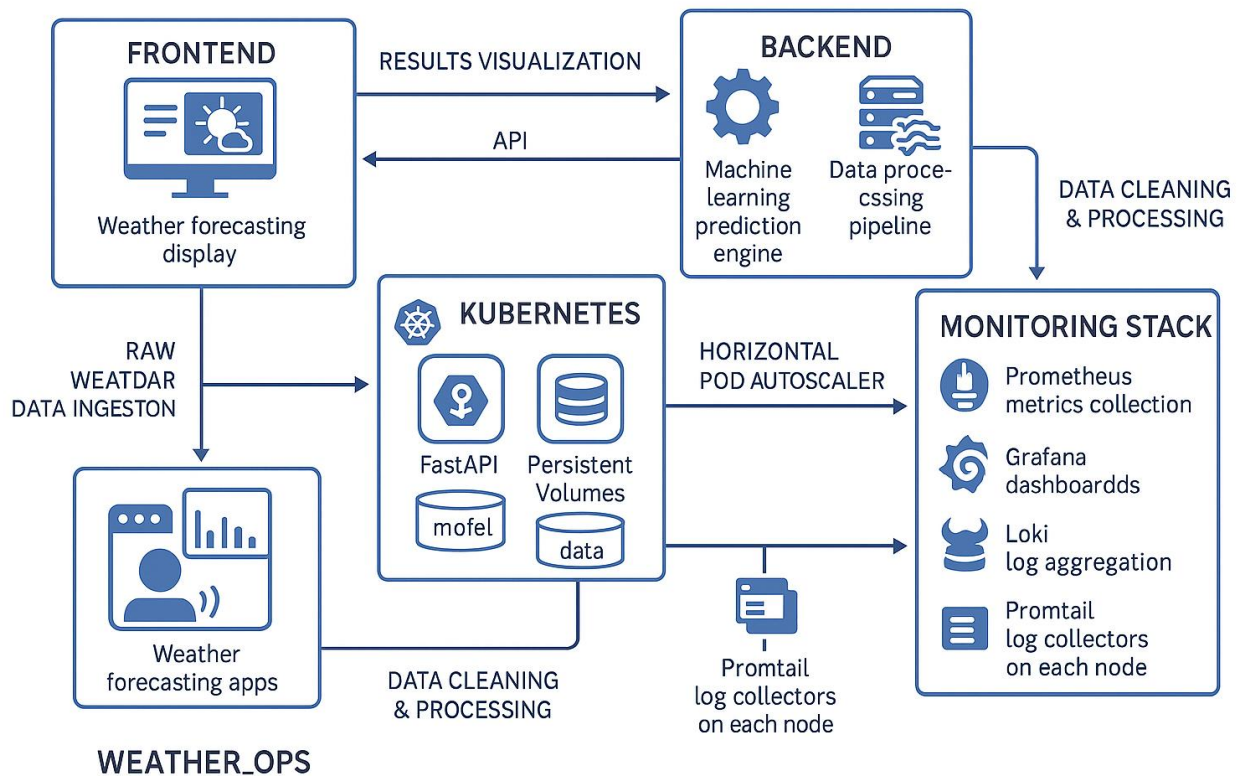


Executive Summary:

Weather_ops is a modern weather forecasting system that leverages machine learning to predict temperature trends based on historical weather data. The system features a microservices architecture with a Streamlit frontend and FastAPI backend, containerized with Docker and orchestrated with Kubernetes, complete with monitoring via Prometheus/Grafana.

1. System Architecture

The Weather_ops system implements a modern cloud-native architecture designed for scalability, resilience, and observability. The architecture consists of these major components:



1.1 Microservices Architecture

Weather_ops is built on a microservices architecture with two primary services:

- **Frontend Service:** A Streamlit-based web application providing intuitive visualization of weather data and predictions
- **Backend Service:** A FastAPI-powered API service handling data processing, model training, and prediction generation

This separation of concerns allows each component to scale independently based on demand and evolve at its own pace.

1.2 Containerization

Both services are containerized using Docker:

- Each service has its own Dockerfile defining its runtime environment and dependencies
- Docker Compose is used for local development and testing
- Container images are versioned and pushed to Docker Hub during the CI/CD process

1.3 Kubernetes Orchestration

The production deployment uses Kubernetes for container orchestration:

- Dedicated Kubernetes namespace (weather-ops) for resource isolation

- Deployment manifests for each service defining replicas, resource limits, and health checks
- Horizontal Pod Autoscaler (HPA) for the backend service to handle varying loads
- Persistent volumes for storing models and datasets
- Service and Ingress resources for network routing

1.4 Data Flow Architecture

The system's data flow follows these paths:

- 1.Raw weather data ingestion via the backend API
- 2.Data cleaning and preprocessing by the backend service
- 3.Model training and saving to persistent storage
- 4.Frontend requests predictions from the backend API
- 5.Results visualization in the frontend UI

1.5 Observability Stack

A comprehensive monitoring and logging stack ensures system observability:

- Metrics Collection: Prometheus scrapes custom application metrics and system resources
- Visualization: Grafana dashboards for metrics visualization and alerting
- Log Aggregation: Loki collects logs from all components
- Log Shipping: Promtail captures and forwards container logs to Loki
- Dashboard: Custom Grafana dashboards for application metrics and logs

1.6 CI/CD Pipeline

A Jenkins-based CI/CD pipeline automates the build, test, and deployment process:

- Source code versioning with Git
- Automated testing with pytest
- Container image building and pushing to Docker Hub
- Kubernetes deployment automation
- Monitoring and logging infrastructure deployment.

2. Technology Stack

The Weather_ops system utilizes a modern technology stack carefully selected to meet the requirements of a scalable, maintainable, and observable weather forecasting platform:

2.1 Frontend Technologies

- Streamlit: Powers the interactive web interface, providing dynamic data visualization and user interaction
- Python: Core programming language for frontend development
- pandas: Data manipulation library for transforming and preparing data for visualization

- Plotly/Matplotlib: Data visualization libraries for creating interactive weather charts and forecast displays

2.2 Backend Technologies

- FastAPI: High-performance web framework for building APIs with automatic documentation
- Python: Core programming language for backend development
- pandas: Used for efficient data manipulation and preprocessing
- scikit-learn: Machine learning library for model development, training, and predictions
- joblib: For model serialization and persistence
- uvicorn: ASGI server for serving FastAPI applications
- pydantic: Data validation and settings management

2.3 DevOps & Infrastructure

- Docker: Container platform for packaging applications and dependencies
- Kubernetes: Container orchestration platform for managing deployments, scaling, and service discovery
- Jenkins: CI/CD platform for automating build, test, and deployment processes
- Minikube: Local Kubernetes environment for development and testing
- kubectl: Command-line tool for interacting with Kubernetes clusters

2.4 Data Pipeline

- DVC (Data Version Control): Version control system specifically for data and models
- Git: Source code version control
- Persistent Volumes: Kubernetes storage for models and datasets

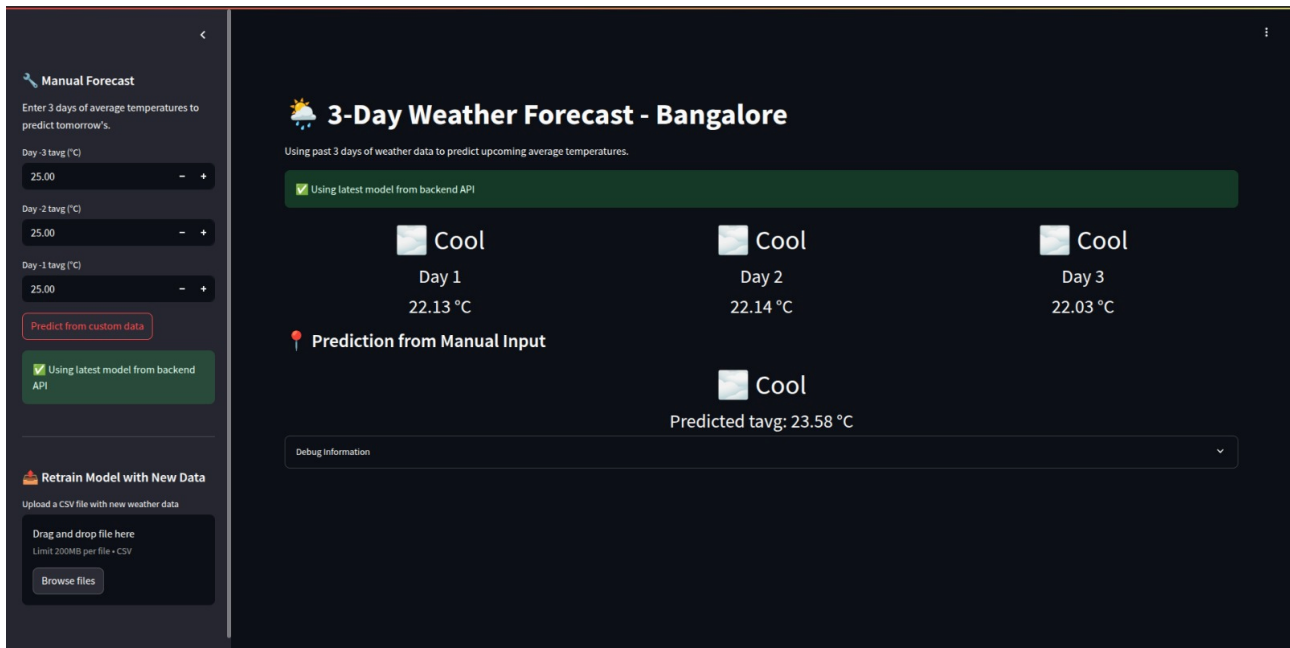
2.5 Monitoring & Logging Infrastructure

- Prometheus: Time-series database for metrics collection and alerting
- Grafana: Visualization platform for metrics and dashboards
- Loki: Log aggregation system designed for Kubernetes environments
- Promtail: Log shipping agent that sends logs to Loki

2.6 Testing & Quality Assurance

- pytest: Python testing framework for unit and integration tests
- pytest-cov: Test coverage reporting tool
- httpx: HTTP client for testing FastAPI endpoints

3. Component Analysis:



1. Frontend Component

The frontend component is built using Streamlit and serves as the user interface for the weather forecasting system.

1.1 Implementation Details

- Technology: Python-based Streamlit application (Python 3.12)
- Containerization: Deployed as a Docker container with exposed port 8501
- Data Access: Mounted volume for accessing shared data files

1.2 Key Features and Capabilities

- 3-day weather forecast display with visual indicators
- Custom forecast tool for manual data input
- Model retraining interface via file upload functionality
- Robust error handling with fallback mechanisms for data and model access

1.3 Integration Points

- Connects to backend via REST API calls
- Fetches model and cleaned data from backend endpoints
- Sends user-uploaded data for model retraining

Key Components

1. Main Application File File: `frontend/app.py`

Purpose: Serves as the main entry point for the frontend application. It contains logic to render the user interface and handle user interactions.

2. Dependencies File: `frontend/requirements.txt`

Purpose: Lists required Python packages for the frontend application. Key Dependencies: `streamlit`: For building the web interface Other libraries for data handling and visualization

3. Frontend Script File: ``frontend/run_frontend.sh`` Purpose: Shell script to start the frontend service. Typically responsible for: Activating the virtual environment Launching the Streamlit application

4. Data Files Directory: ``frontend/data/``

Files: ``cleaned_weather.csv``: Preprocessed weather data used for display or analysis

``model.pkl``: Serialized machine learning model used for predictions

5. Docker Support File: ``frontend/Dockerfile``

Purpose: Provides instructions for building a Docker image of the frontend. It: Sets up the runtime environment Runs the application inside a container

Running the Frontend

Option 1: Using the Shell Script Execute the ``run_frontend.sh`` script to launch the application locally.

Option 2: Using Docker Build and run the Docker container using the ``frontend/Dockerfile``:
`bash docker build -t weather_frontend ./frontend docker run -p 8501:8501 weather_frontend.`

2. Backend Component

The backend component is built using FastAPI and serves as the API and data processing engine for the weather forecasting system.

2.1 Implementation Details

- Technology: Python-based FastAPI application (Python 3.12)
- Containerization: Deployed as a Docker container with exposed port 5000
- Data Storage: Persistent storage for weather data and trained models
- Model Management: Automated training pipeline with version control

2.2 Key Features and Capabilities

- REST API endpoints for data retrieval and prediction
- Machine learning model training and management
- Data processing and cleaning pipeline
- Metrics collection for monitoring and performance analysis
- Structured logging for operational visibility

2.3 Integration Points

- Exposes REST API for frontend consumption
- Provides endpoints for raw and cleaned data access
- Delivers model prediction functionality
- Accepts new data for model retraining

2.4 API Design & Endpoints

- `/health`: Health check endpoint for monitoring
- `/metrics`: Prometheus metrics endpoint for monitoring

- `/model`: Returns metadata about the current ML model
- `/predict`: Core prediction endpoint that accepts parameters and returns forecast
- `/train`: Endpoint to trigger model retraining with new data
- `/data`: Endpoint to access cleaned weather data
- `/raw-data`: Endpoint to access raw weather data
- `/upload-data`: Endpoint to receive new weather data files

2.5 Data Processing Pipeline

- Data Acquisition: Handled by `data_getter.py`, which retrieves data from CSV files or uploads
- Data Cleaning: Implemented in `data_cleaner.py`, performs:
 - Missing value imputation
 - Outlier detection and removal
 - Feature normalization
 - Date formatting and time series preparation
- Data Validation: Enforces schema validation and data quality checks
- Data Versioning: Uses DVC (Data Version Control) to track data changes

2.6 Machine Learning Implementation

- Model Selection: Uses time series forecasting models from `scikit-learn`
- Feature Engineering: Creates time-based features for improved prediction
- Training Process: Implemented in `train.py`:
 - Data splitting into train/validation sets
 - Hyperparameter optimization
 - Model evaluation using RMSE and MAE metrics
- Model Persistence: Saves trained models with `joblib` serialization
- Model Versioning: Tracks model versions with metadata in `model_version_info.json`

2.7 Performance & Scalability

- Asynchronous Processing: Uses FastAPI's `async` capabilities for non-blocking operations
- Resource Optimization: Configurable worker processes based on available resources
- Horizontal Scaling: Supports scaling via Kubernetes HPA (Horizontal Pod Autoscaler)
- Caching: Implements response caching for frequently requested predictions
- Batching: Supports batch prediction for improved throughput

2.8 Security Measures

- Input Validation: Strict schema validation using Pydantic models
- Rate Limiting: Protects against excessive API calls
- Error Handling: Structured error responses with appropriate HTTP status codes
- Logging: Comprehensive logging with sensitive data masking
- Containerization: Isolated runtime environment with minimal dependencies

2.9 Testing Strategy

- Unit Tests: Individual component testing in /tests directory
- Integration Tests: API endpoint testing with simulated requests
- Data Validation Tests: Ensures data processing pipeline integrity
- Model Quality Tests: Validates model prediction accuracy
- CI/CD Integration: Automated test execution in the Jenkins pipeline

2.10 Monitoring & Observability

- Custom Metrics: Exposed via /metrics endpoint in Prometheus format
- Key Metrics Tracked:
 - Prediction request count and latency
 - Model error rates (RMSE, MAE)
 - Training durations
 - Data processing statistics
- Structured Logging: JSON-formatted logs with contextual information
- Health Checks: Active and passive monitoring of service health

Key Components

1. Main Application File

- File: main.py
- Purpose: Serves as the entry point for the FastAPI application, defining API routes and handling HTTP requests
- Features: RESTful API implementation, dependency injection, request validation, response formatting

2. Prediction Module

- File: predict.py
- Purpose: Contains the logic for making weather predictions using trained models
- Features: Model loading, prediction generation, confidence scoring

3. Training Module

- File: train.py
- Purpose: Implements model training functionality
- Features: Dataset preparation, algorithm selection, hyperparameter tuning, model evaluation

4. Data Processing Modules

- Files: data_cleaner.py, data_getter.py
- Purpose: Handle data acquisition, cleaning, and preprocessing
- Features: Data validation, missing value handling, feature engineering, outlier detection

5. Monitoring Module

- File: metrics.py
- Purpose: Collects and exposes application metrics

- Features: Custom metrics for model performance, API usage tracking, resource utilization

6. Dependencies

- File: requirements.txt
- Purpose: Lists required Python packages
- Key Dependencies:
 - fastapi: Web framework for building APIs
 - uvicorn: ASGI server implementation
 - pandas: For data manipulation
 - scikit-learn: For machine learning capabilities
 - prometheus-client: For metrics collection

7. Data Files

- Directory: data
- Files:
 - raw_weather.csv: Original weather data
 - cleaned_weather.csv: Preprocessed weather data
 - model.pkl: Serialized machine learning model

8. Testing

- Directory: tests
- Files:
 - test_main.py: Tests for API endpoints
 - test_predict.py: Tests for prediction functionality
 - test_data_cleaner.py: Tests for data processing

9. Docker Support

- File: Dockerfile
- Purpose: Provides instructions for building a Docker image
- Features: Multi-stage build process, dependencies installation, runtime configuration

Running the Backend

Option 1: Directly with Python

```
cd backend
```

```
pip install -r requirements.txt
uvicorn main:app --reload
```

Option 2: Using Docker

```
docker build -t weather_backend ./backend
docker run -p 5000:5000 weather_backend
```

Option 3: Using Kubernetes

kubectl apply -f kubernetes/backend.yaml

The backend exposes a Swagger UI for API documentation at /docs and ReDoc at /redoc for more detailed documentation. These interfaces allow developers to interactively explore and test the available endpoints.

3. Data Processing Pipeline

The data processing pipeline is responsible for acquiring, cleaning, transforming, and preparing weather data for the machine learning models.

3.1 Implementation Details

- Technology: Python-based data processing using pandas (as seen in data_cleaner.py)
- Storage: Raw and processed data stored in CSV format in the data directory
- Versioning: DVC (Data Version Control) for tracking data changes (evident from dvc.yaml and dvc.lock files)
- Integration: Connected with both frontend and backend components via shared data files

3.2 Key Features and Capabilities

- Data acquisition from files (as implemented in data_getter.py)
- Data cleaning and preprocessing through the data_cleaner.py module
- Version control for tracking data modifications
- Support for both raw and cleaned data formats

3.3 Integration Points

- Backend access to data files for model training and predictions
- Frontend access to cleaned data for visualization
- DVC integration for data versioning
- Test suite for validating data processing functionality

3.4 Data Flow Structure

- Raw data stored in raw_weather.csv
- Processing through data_cleaner.py
- Cleaned data output to data/cleaned_weather.csv
- Data accessed by model training module (train.py)
- Processed data used for predictions (predict.py)

3.5 Data Cleaning Operations

- Implemented in data_cleaner.py with functionality for:
 - Data format standardization
 - Missing value handling
 - Data type conversions
 - Feature preparation for model consumption

Key Components

1.Data Cleaning Module

- File: `data_cleaner.py`
- Purpose: Processes raw weather data into clean, model-ready format
- Features: Data transformation, format standardization, quality improvement

2.Data Acquisition Module

- File: `data_getter.py`
- Purpose: Retrieves weather data from file sources
- Features: File reading, data loading, initial validation

3.Data Version Control

- Files: `dvc.yaml`, `dvc.lock`
- Purpose: Tracks changes to data files over time
- Features: Version history, reproducibility, data lineage

4.Data Storage

- Directories: `data`, `data`, `data`
- Purpose: Stores raw and processed data files
- Key Files:
 - `raw_weather.csv`: Original weather data
 - `cleaned_weather.csv`: Preprocessed weather data ready for models

5.Testing Framework

- File: `tests/test_data_cleaner.py`
- Purpose: Validates data processing functionality
- Features: Unit tests for data cleaning operations

Running the Data Pipeline

Option 1: Via Backend API The data pipeline is typically triggered through the backend API, which manages the data processing workflow.

Option 2: Direct Execution

cd backend

```
python -c "from data_cleaner import clean_data; clean_data('data/raw_weather.csv',  
'data/cleaned_weather.csv')"
```

or

dvc repro

can also trigger the dvc pipeline

Option 3: As Part of Model Training The data processing is automatically invoked during model training, as implemented in the `train.py` module.

The data processing pipeline ensures that weather data is properly prepared for machine learning models, maintaining consistency between training and prediction phases while preserving data quality and format standards.

4. Machine Learning Implementation

The machine learning component provides the core prediction capabilities of the Weather_ops system through time-series forecasting models.

4.1 Implementation Details

- Technology: Python-based machine learning using scikit-learn
- Model Storage: Serialized models saved using joblib in model.pkl files
- Version Tracking: Model versioning through model_version_info.json
- Deployment: Models deployed as part of the backend container

4.2 Key Features and Capabilities

- Time-series forecasting for weather predictions
- Model training with hyperparameter optimization
- Model evaluation and performance metrics
- Version control for model artifacts
- Prediction generation with confidence intervals

4.3 Integration Points

- Trained models consumed by prediction API endpoints
- Training pipeline triggered through API or scheduled jobs
- Model metrics exposed to monitoring system
- Frontend visualization of prediction results

4.4 Model Architecture

- Time-series regression models for weather forecasting
- Feature engineering for temporal data patterns
- Ensemble techniques for improved prediction stability
- Cross-validation for robust performance estimation

4.5 Training Process

- Data preparation with train/validation splits
- Feature scaling and normalization
- Model selection based on performance metrics
- Hyperparameter tuning for optimization
- Evaluation using RMSE and MAE metrics

4.6 Prediction Pipeline

- Model loading from serialized format

- Data preprocessing for new inputs
- Prediction generation for requested timeframes
- Confidence interval calculation
- Response formatting for API consumption

4.7 Performance Monitoring

- Tracking prediction accuracy over time
- Drift detection for model performance
- Automated metrics logging in metrics.json
- Integration with Prometheus for real-time monitoring

Key Components

1.Training Module

- File: train.py
- Purpose: Implements the model training pipeline
- Features: Data preparation, model fitting, hyperparameter tuning, performance evaluation

2.Prediction Module

- File: predict.py
- Purpose: Handles prediction generation using trained models
- Features: Model loading, prediction calculation, confidence scoring

3.Model Versioning

- File: model_version_info.json
- Purpose: Tracks model versions and metadata
- Features: Version history, training parameters, performance metrics

4.Model Storage

- Files: model.pkl, model.pkl, model.pkl
- Purpose: Stores serialized machine learning models
- Features: Portable model format accessible to all components

5.Metrics Tracking

- File: metrics.json
- Purpose: Records model performance metrics
- Features: Accuracy tracking, training statistics, prediction performance

6.Testing

- File: test_predict.py
- Purpose: Validates prediction functionality
- Features: Unit tests for model loading, prediction accuracy, error handling

Running the ML Pipeline

Option 1: Model Training

```
cd backend
python train.py
```

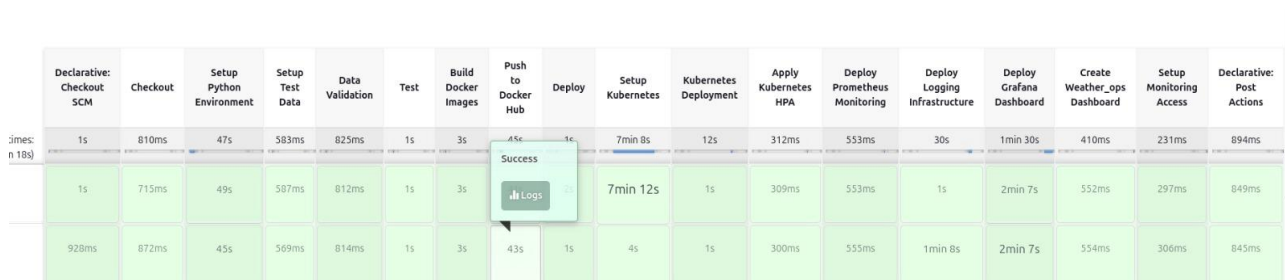
Option 2: Via API Endpoint Training can be triggered through the /train API endpoint, which will process available data and update the model.

Option 3: Kubernetes Job

```
kubectrl apply -f kubernetes/model-generator.yaml
```

The machine learning implementation ensures accurate weather predictions while maintaining version control of models and monitoring prediction performance through integration with the monitoring stack.

5. CI/CD Implementation



The Weather_ops project uses a robust CI/CD pipeline implemented with Jenkins to automate testing, building, and deployment of the application.

5.1 Implementation Details

- Technology: Jenkins CI/CD server
- Configuration: Declarative Jenkinsfile pipeline
- Containerization: Docker for build and deployment
- Orchestration: Kubernetes deployment automation
- Infrastructure as Code: Pipeline and deployment configurations in version control

5.2 Key Features and Capabilities

- Automated testing of backend components
- Containerized build process with Docker
- Image versioning and Docker Hub integration
- Kubernetes deployment automation
- Monitoring infrastructure deployment
- Parameterized deployments with safety controls

5.3 Integration Points

- Source code repository for code changes
- Docker Hub for container image storage
- Kubernetes cluster for application deployment
- Monitoring stack for operational visibility
- Testing framework for quality assurance

5.4 Pipeline Stages

- Code checkout from version control
- Python environment setup
- Test data preparation
- Data validation
- Unit and integration testing
- Docker image building
- Container registry publishing
- Kubernetes deployment
- Monitoring infrastructure setup
- Grafana dashboard provisioning

5.5 Deployment Strategy

- Environment-specific deployments (using parameters)
- Kubernetes namespace isolation
- Rolling updates for zero-downtime deployment
- Configurable deployment options via parameters
- Proper resource cleanup and management

5.6 Infrastructure Automation

- Kubernetes resource provisioning
- Monitoring stack deployment
- Logging infrastructure setup
- Dashboard configuration
- Service exposure and access management

5.7 Safety Measures

- FORCE_K8S_DEPLOY parameter to prevent accidental deployments
- Test validation before deployment
- Docker credential management
- Resource cleanup on pipeline completion
- Detailed logging for troubleshooting

Key Components

1. Pipeline Definition

- File: Jenkinsfile
- Purpose: Defines the CI/CD pipeline stages and steps
- Features: Multi-stage pipeline, environment configuration, Docker integration

2. Kubernetes Deployment Scripts

- File: deploy_kubernetes.sh
- Purpose: Automates Kubernetes resource deployment
- Features: Namespace creation, application deployment, service setup

3. Logging Deployment Script

- File: deploy_logging.sh
- Purpose: Sets up the logging infrastructure
- Features: Loki, Promtail, and Grafana configuration

4.Cleanup Script

- File: cleanup_kubernetes.sh
- Purpose: Removes deployed resources from Kubernetes
- Features: Resource cleanup, namespace management

5.Docker Control Script

- File: docker-control.sh
- Purpose: Manages Docker container operations
- Features: Container building, starting, stopping

6.Jenkins Setup Scripts

- Files: setup_jenkins.sh, setup_jenkins_k8s.sh, setup_jenkins_k8s_simple.sh
- Purpose: Configure Jenkins for the pipeline
- Features: Jenkins installation, plugin setup, credential configuration

Running the CI/CD Pipeline

Option 1: Jenkins UI Trigger the pipeline through the Jenkins web interface, optionally setting parameters like FORCE_K8S_DEPLOY.

Option 2: Jenkins CLI

```
jenkins-cli build Weather_ops -p FORCE_K8S_DEPLOY=true
```

Option 3: Webhook Trigger Pipeline can be automatically triggered by repository webhooks when new code is pushed.

6. Security Measures

The Weather_ops system implements several security practices to protect data and ensure operational integrity.

6.1 Container Security

- Containerized Environments: Both frontend and backend services run in isolated Docker containers
- Defined Resource Limits: Kubernetes deployments include CPU and memory limits to prevent resource exhaustion
- Container Isolation: Services run in separate containers with specific port exposures

6.2 Network Security

- Namespace Isolation: Application components deployed in dedicated "weather-ops" namespace
- Service Discovery: Internal Kubernetes service discovery for secure inter-service communication
- NodePort Services: Controlled external access to services like Grafana

6.3 API Security

- Input Validation: FastAPI's Pydantic models validate input data formats (as seen in main.py)
- Error Handling: Structured error responses in API endpoints
- Defined API Endpoints: Clear API surface with specifically defined endpoints

6.4 Data Protection

- Data Versioning: DVC (Data Version Control) for tracking data changes
- Persistent Volume Claims: Kubernetes PVCs for protecting application data
- Restricted Data Access: Data access managed through Kubernetes volume mounts

6.5 Authentication & Authorization

- Grafana Authentication: Basic authentication for accessing monitoring dashboards
- RBAC for Logging: Role-Based Access Control for Promtail via promtail-rbac.yaml
- Service Account Permissions: Specific service accounts for monitoring components

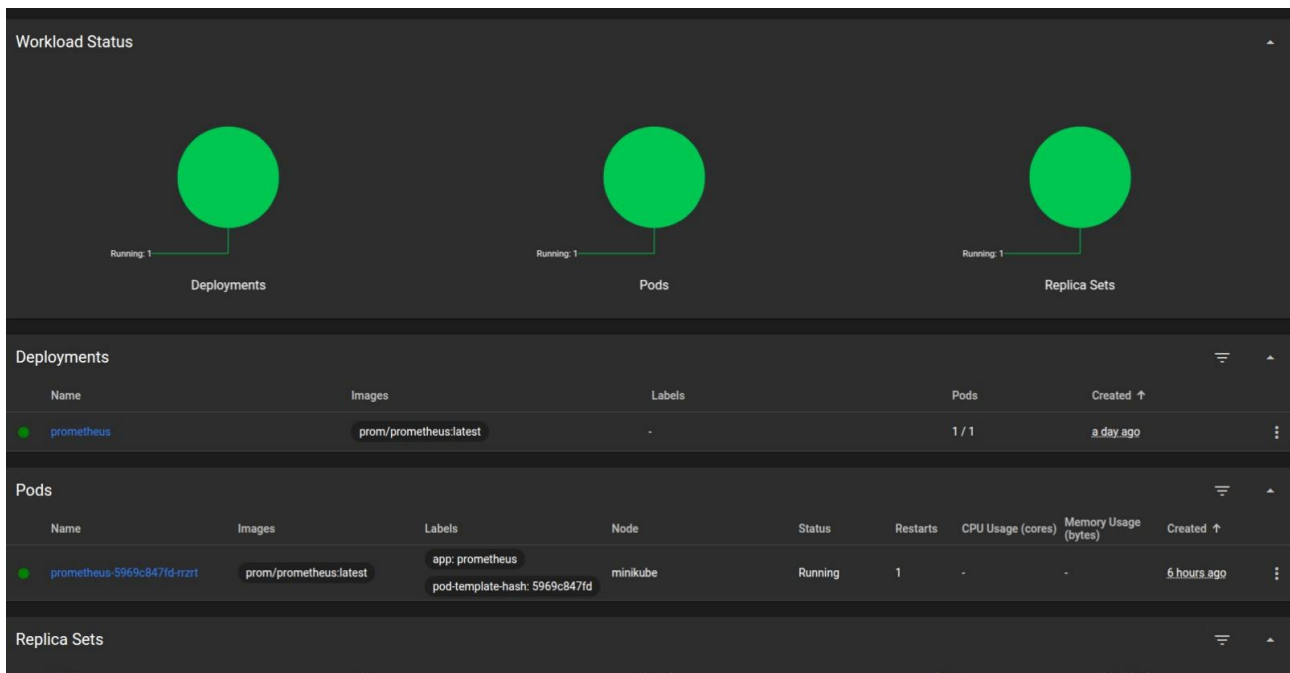
6.6 CI/CD Security

- Credential Management: Docker Hub credentials securely stored in Jenkins
- Deployment Controls: FORCE_K8S_DEPLOY parameter to prevent accidental deployments
- Pipeline Environment Isolation: Jenkins pipeline runs in controlled environment

6.7 Monitoring & Observability

- Centralized Logging: Log aggregation with Loki and Promtail
- Application Metrics: Prometheus metrics collection for detecting unusual behavior
- Visual Dashboards: Grafana dashboards for monitoring system health
- Log Filtering: Capability to filter logs by container/service for focused analysis

7. Kubernetes Deployment and Orchestration



The Weather_ops system utilizes Kubernetes for container orchestration, providing scalability, resilience, and automated management of the application components.

7.1 Implementation Details

- Namespace: Dedicated "weather-ops" namespace for resource isolation
- Resource Organization: Complete set of Kubernetes manifests in the kubernetes directory
- Deployment Automation: Scripts for consistent application deployment
- Monitoring Integration: Prometheus and Grafana deployed as Kubernetes resources
- Logging Stack: Loki and Promtail configured for container log collection

7.2 Key Features and Capabilities

- Separate deployments for frontend and backend components
- Horizontal Pod Autoscaling (HPA) for backend service
- Persistent storage for data and models
- ConfigMaps for application configuration
- Comprehensive monitoring and logging infrastructure
- Resource limits and requests for optimal performance

7.3 Core Application Components

- Backend Deployment: FastAPI service with ML capabilities
 - Resource limits and requests
 - Health check probes
 - Environment variable configuration
 - Service for network access
- Frontend Deployment: Streamlit UI service
 - Container configuration
 - Service exposure

- Backend connectivity
- Persistent Storage: Volumes for data retention
 - PersistentVolume and PersistentVolumeClaim
 - Mounted to appropriate pods

7.4 Scaling and Resilience

- Horizontal Pod Autoscaler: Automatic scaling based on CPU utilization
- Resource Quotas: Defined limits to prevent resource exhaustion
- Liveness and Readiness Probes: Health checking for automatic recovery
- Deployment Strategies: Rolling updates for zero-downtime deployments

7.5 Monitoring Infrastructure

- Prometheus: Metrics collection and storage
 - Configured via ConfigMap
 - Deployed as a StatefulSet
 - Service for network access
- Grafana: Visualization and dashboarding
 - Dashboard provisioning
 - Data source configuration
 - Service exposure via NodePort

7.6 Logging Stack

- Loki: Log aggregation service
 - Centralized log storage
 - Query capabilities
- Promtail: Log collection agent
 - DaemonSet deployment for cluster-wide coverage
 - RBAC configuration for log access
 - ConfigMap for log processing rules

Key Components

1.Core Application Manifests

- Files: backend.yaml, frontend.yaml, namespace.yaml
- Purpose: Define the primary application components
- Features: Deployments, Services, configuration

2.Autoscaling Configuration

- File: backend-hpa.yaml
- Purpose: Enables automatic scaling of the backend service
- Features: CPU-based scaling, min/max replica configuration

3.Storage Configuration

- File: persistent-volume.yaml
- Purpose: Provides persistent storage for the application
- Features: Volume definition, access modes, capacity

4.Configuration Management

- File: configmap.yaml
- Purpose: Manages application configuration
- Features: Environment-specific settings, feature flags

5.Model Generation Job

- File: model-generator.yaml
- Purpose: Kubernetes Job for model training/generation
- Features: Batch processing, resource allocation

6.Network Access

- File: ingress.yaml
- Purpose: External access configuration
- Features: Routing rules, host configuration

7.Monitoring Configuration

- Files: Files in prometheus and grafana directories
- Purpose: Set up the monitoring infrastructure
- Features: Metrics collection, dashboard configuration

8.Logging Infrastructure

- Files: Files in loki and promtail directories
- Purpose: Configure the logging stack
- Features: Log collection, storage, and querying

9.Deployment Scripts

- Files: deploy_kubernetes.sh, cleanup_kubernetes.sh
- Purpose: Automate deployment and cleanup
- Features: Sequential resource creation, proper dependency handling

Deployment Process

Option 1: Using the Deployment Script

```
./deploy_kubernetes.sh
```

Option 2: Manual Deployment

```
kubectl apply -f kubernetes/namespace.yaml
```

```
kubectl apply -f kubernetes/persistent-volume.yaml
```

```
kubectl apply -f kubernetes/configmap.yaml
```

```
kubectl apply -f kubernetes/backend.yaml
```

```
kubectl apply -f kubernetes/frontend.yaml
```

```
kubectl apply -f kubernetes/backend-hpa.yaml
```

```
kubectl apply -f kubernetes/ingress.yaml
```

Option 3: CI/CD Pipeline The Jenkins pipeline can deploy the application automatically when triggered.



8.2 Key Features and Capabilities

- Real-time metrics collection from application and infrastructure
- Custom application metrics for business intelligence
- Centralized log aggregation with structured searching
- Visual dashboards for operational insights
- Separate visualizations for metrics and logs
- Prometheus metrics endpoint exposure in backend service

8.3 Prometheus Setup

- Deployment Configuration: Standard Kubernetes deployment
- Configuration: ConfigMap-based scrape configuration
- Service: NodePort service for external access
- Targets: Backend application and Kubernetes components
- Metrics Paths: Standard /metrics endpoints

8.4 Grafana Configuration

- Dashboards: Pre-configured dashboards for application metrics and logs
- Data Sources: Prometheus and Loki configured as data sources
- Access: NodePort service for external dashboard access
- Dashboard Provisioning: Automatic dashboard setup via ConfigMaps
- Persistence: Configuration for dashboard and data source persistence

8.5 Loki Log Aggregation

- Deployment: Kubernetes deployment with appropriate storage
- Integration: Seamlessly integrated with Grafana for visualization
- Log Format: Structured log collection with metadata
- Query Capabilities: Log filtering by container, service, and content

8.6 Promtail Log Collection

- Deployment Model: DaemonSet to run on every Kubernetes node
- RBAC Configuration: Proper permissions for log access
- Target Configuration: Container logs in the weather-ops namespace
- Pipeline Stages: Log parsing, labeling, and formatting
- Log Forwarding: Direct forwarding to Loki service

8.7 ELK Stack Alternative

- Components: Elasticsearch, Kibana, and Filebeat
- Configuration: Docker Compose setup for local development
- Filebeat: Kubernetes configuration for production deployment
- Integration: Alternative logging pipeline alongside Loki

Key Components

1.Prometheus Configuration

- File: prometheus-configmap.yaml
- Purpose: Defines metric scrape targets and intervals
- Features: Job configurations, scrape intervals, metric paths

2.Prometheus Deployment

- File: prometheus-deployment.yaml
- Purpose: Deploys Prometheus server in Kubernetes
- Features: Container configuration, resource limits, storage

3.Prometheus Service

- File: prometheus-service.yaml
- Purpose: Exposes Prometheus for access
- Features: NodePort configuration, port mapping

4.Grafana Deployment

- Files: grafana-deployment.yaml, grafana-fixed-deployment.yaml
- Purpose: Deploys Grafana visualization platform
- Features: Container configuration, volume mounts for dashboards

5.Grafana Dashboard Provider

- File: grafana-dashboard-provider.yaml
- Purpose: Configures dashboard provisioning
- Features: Dashboard source configuration, refresh settings

6.Grafana Data Sources

- File: grafana-datasources.yaml
- Purpose: Pre-configures Prometheus and Loki as data sources
- Features: Data source definitions, access modes

7.Grafana Service

- File: grafana-service.yaml
- Purpose: Exposes Grafana UI
- Features: NodePort configuration, port mapping

8.Log Dashboard

- File: weather-ops-logs-dashboard.json
- Purpose: Pre-configured dashboard for log visualization
- Features: Log panels, filtering options, refresh settings

9.Loki Deployment

- File: loki-deployment.yaml
- Purpose: Deploys Loki log aggregation service
- Features: Container configuration, storage settings

10.Promtail Configuration

- File: promtail-configmap.yaml
- Purpose: Defines log collection rules
- Features: Pipeline stages, label extraction, log sources

11.Promtail Deployment

- File: promtail-deployment.yaml
- Purpose: Deploys Promtail as a DaemonSet
- Features: Volume mounts for logs, configuration

12.Promtail RBAC

- File: promtail-rbac.yaml
- Purpose: Provides necessary permissions for log access
- Features: ServiceAccount, ClusterRole, ClusterRoleBinding

13.Application Metrics

- File: metrics.py
- Purpose: Defines and exposes application-specific metrics
- Features: Custom metrics, Prometheus integration

14.Filebeat Configuration

- Files: Files in filebeat directory
- Purpose: Alternative log collection with ELK stack
- Features: Log shipping configuration, RBAC setup

Deployment Process

Option 1: Deploying Prometheus and Grafana

```
kubectl apply -f kubernetes/prometheus/
```

```
kubectl apply -f kubernetes/grafana/
```

Option 2: Deploying Loki and Promtail

```
./deploy_logging.sh
```

Option 3: Deploying Filebeat

```
./deploy_filebeat.sh
```

9. Testing Strategy

The Weather_ops project implements a comprehensive testing approach to ensure the reliability and correctness of its components, particularly focusing on the backend functionality.

9.1 Implementation Details

- Framework: pytest for Python unit and integration testing
- Location: Tests organized in the tests directory
- Focus Areas: Data processing, API endpoints, prediction functionality
- Integration: Tests run automatically in the CI/CD pipeline
- Coverage: Key components covered with dedicated test modules

9.2 Key Features and Capabilities

- Unit testing of core functions and classes
- API endpoint testing with simulated requests
- Data validation testing for the data processing pipeline
- Model prediction verification
- Error handling validation

- Isolated test environment to prevent side effects

9.3 Test Categories

- Data Processing Tests: Validating the data cleaning and transformation logic
- API Endpoint Tests: Ensuring correct behavior of FastAPI routes
- Prediction Tests: Verifying machine learning model functionality
- Error Handling Tests: Confirming proper error responses for edge cases
- Integration Tests: Testing component interaction within the backend

9.4 Testing Approach

- Fixture-based Testing: Reusable test fixtures for common scenarios
- Parameterized Tests: Multiple scenarios with different inputs
- Mocking: Isolation of components for targeted testing
- Assertion-based Validation: Clear pass/fail criteria for each test
- Automated Execution: Tests run as part of the CI/CD pipeline

9.5 Test Data Management

- Test Fixtures: Sample data created specifically for tests
- Isolated Test Environment: Tests don't affect production data
- Representative Data Sets: Test data that reflects real-world scenarios
- Edge Cases: Testing boundary conditions and unusual inputs

Key Components

1. Data Cleaning Tests

- File: `test_data_cleaner.py`
- Purpose: Tests data transformation and cleaning functions
- Features: Validation of data preprocessing, format conversion, integrity checks

2. API Endpoint Tests

- File: `test_main.py`
- Purpose: Tests FastAPI routes and responses
- Features: HTTP request simulation, response validation, error handling verification

3. Prediction Module Tests

- File: `test_predict.py`
- Purpose: Tests machine learning prediction functionality
- Features: Model loading, prediction accuracy, input handling

4. Test Configuration

- Integration: pytest configured in the CI/CD pipeline
- Execution: Tests run before deployment to ensure quality

Running the Tests:

Option 1: Local Test Execution

`cd backend`

`pytest -v`

Option 2: Specific Test Module

`cd backend`

`pytest tests/test_data_cleaner.py -v`

Option 3: CI/CD Pipeline Tests run automatically during the Jenkins pipeline execution in the 'Test' stage.

10. Future Enhancements

The Weather_ops system has a solid foundation for weather forecasting, but several potential enhancements could further improve functionality, scalability, and user experience.

10.1 Machine Learning Improvements

- **Advanced Models:** Implement deep learning models (LSTM, transformer architecture) for improved prediction accuracy
- **Ensemble Methods:** Combine multiple prediction models to reduce forecast error
- **Feature Expansion:** Incorporate additional weather features (humidity, atmospheric pressure, wind direction)
- **Transfer Learning:** Leverage pre-trained models for more accurate initial predictions
- **Online Learning:** Implement continuous model updating with new data

10.2 Data Processing Enhancements

- **Real-time Data Sources:** Integrate with weather APIs for live data feeds
- **Geographic Expansion:** Support multiple geographic regions with region-specific models
- **Historical Data Analysis:** Add long-term trend analysis and climate change indicators
- **Data Quality Metrics:** Implement automated data quality scoring and reporting
- **Anomaly Detection:** Build system to identify unusual weather patterns

10.3 Architecture Improvements

- **Microservices Refinement:** Further decompose backend into specialized microservices
- **Event-driven Architecture:** Implement message queues for asynchronous processing
- **GraphQL API:** Add GraphQL support for more flexible data queries
- **WebSocket Support:** Enable real-time updates for the frontend
- **Edge Deployment:** Enable deployment to edge locations for reduced latency

10.4 Monitoring and Observability

- **Distributed Tracing:** Implement OpenTelemetry for request tracing across components
- **Business Metrics:** Track and visualize forecast accuracy over time
- **Alert System:** Configure intelligent alerting based on prediction anomalies

- User Behavior Analytics: Track feature usage and common prediction patterns
- Synthetic Monitoring: Implement automated testing of the complete system

10.5 Frontend Experience

- Mobile Responsiveness: Optimize UI for mobile devices
- Interactive Maps: Geographical visualization of weather predictions
- Customizable Dashboards: Allow users to create personalized forecast views
- Comparison Tools: Side-by-side comparison of different prediction models
- Notification System: Alerts for significant weather changes or events

10.6 Operational Improvements

- Multi-region Deployment: Deploy services across multiple cloud regions for redundancy
- Automated Canary Deployments: Implement progressive rollouts for safer updates
- Cost Optimization: Implement autoscaling based on time-of-day patterns
- Disaster Recovery: Comprehensive backup and recovery procedures
- Chaos Engineering: Test system resilience with controlled failure scenarios

10.7 Security Enhancements

- OAuth Integration: Implement proper authentication system
- Fine-grained Authorization: Role-based access controls for different system aspects
- Security Scanning: Integrate vulnerability scanning in CI/CD pipeline
- Network Security Policies: Implement strict pod-to-pod communication rules
- Secrets Management: Use dedicated secrets management solution

10.8 Testing Expansion

- End-to-End Tests: Implement automated testing of the complete system
- Performance Testing: Add load tests to verify system under high demand
- Chaos Testing: Verify system behavior during component failures
- UI Testing: Automated tests for frontend interactions
- Service Contract Testing: Ensure API compatibility between versions

11. Conclusion

The Weather_ops project represents a comprehensive implementation of a modern weather forecasting system that leverages machine learning, containerization, orchestration, and advanced monitoring techniques.

11.1 Project Achievements

- End-to-End Solution: Successfully implemented a complete weather forecasting system from data processing to visualization
- Microservices Architecture: Built a modular system with separate frontend and backend components

- **Machine Learning Integration:** Developed time-series forecasting models for weather prediction
- **Cloud-Native Design:** Created a fully containerized, Kubernetes-orchestrated application
- **Comprehensive Monitoring:** Established a robust monitoring and logging infrastructure
- **CI/CD Implementation:** Automated the build, test, and deployment process
- **Data Management:** Implemented proper data processing pipelines with version control

11.2 Technical Highlights

- **Frontend:** Streamlit-based interactive user interface for weather data visualization
- **Backend:** FastAPI service with RESTful endpoints for data and prediction access
- **Data Processing:** Robust pipeline for cleaning and preparing weather data
- **Machine Learning:** Trained models for weather forecasting with versioning
- **DevOps Practices:** Docker containerization and Kubernetes orchestration
- **Observability Stack:** Prometheus, Grafana, Loki, and Promtail for complete system visibility
- **Testing Coverage:** Comprehensive test suite for backend components

11.3 Challenges Overcome

- **Data Quality Management:** Implemented cleaning procedures for weather data
- **Model Versioning:** Created a system for tracking model versions and performance
- **Kubernetes Configuration:** Established proper resource allocation and scaling
- **Monitoring Integration:** Connected application metrics to visualization tools
- **Log Aggregation:** Solved log collection and centralization challenges
- **Pipeline Automation:** Created a reliable CI/CD process with appropriate safeguards

11.4 Lessons Learned

- **Infrastructure as Code:** Maintaining Kubernetes configurations in version control proved essential
- **Modular Design:** Separation of concerns facilitated independent component development
- **Testing Importance:** Comprehensive test coverage ensured system reliability
- **Observability First:** Early implementation of monitoring provided valuable insights
- **Automation Benefits:** CI/CD pipeline significantly improved development velocity
- **Resource Management:** Properly configured resource limits prevented cluster issues

11.5 Core Principles Applied

- **Cloud-Native Development:** Embraced containerization and orchestration
- **DevOps Methodology:** Integrated development and operations with automated pipelines
- **Microservices Architecture:** Built loosely coupled, independently deployable services
- **Data-Driven Approach:** Based predictions on historical weather data analysis

- Observability Culture: Prioritized monitoring and logging throughout the system
- Security Mindfulness: Implemented appropriate security measures at multiple levels

GitHub Link: https://github.com/girishofficial/Weather_ops
